

SpinsolveExpert – Pulse Programming Guide

Version 1.40 March 2020

Table of Contents

1	Introduction	2
2	File organisation.....	4
3	The Proton experiment examined	6
3.1	The Pulse Program Macro.....	8
3.2	The interface description.....	9
3.3	The relationships table	11
3.4	The group list	12
3.5	The variable list.....	13
3.6	The pulse program	13
3.7	The phase cycle matrix	13
3.8	The returned variables.....	14
4	Writing a new pulse program	14
5	The Experiment Control Macro	15
5.1	The execpp (execute pulse program) procedure arguments	17
5.2	The execpp procedure in detail	19
5.3	Modifying the execpp procedure to vary the pulse duration	21
Appendix A	Pulse program commands	28
A.1	Acquire NMR data.....	28
A.2	Clear the DSP data memory.....	29
A.3	Delay for a specified time	29
A.4	Frequency modifications	29
A.5	Looping.....	30
A.6	RF pulse production	31
A.7	Shim and gradient commands	32
A.8	Table commands.....	33
Appendix B	Available control groups	34
B.1	Acquisition	34
B.2	AcquisitionBW.....	34
B.3	AcquisitionBW2D	35
B.4	AcquisitionFIR	35
B.5	Display_Std.....	36
B.6	Display_2D	36
B.7	Display_Hz.....	36
B.8	File_Settings.....	37
B.9	Jres_processing.....	37
B.10	Processing_std	37
B.11	Processing_autophase	37
B.12	Processing_filter_autophase	38
B.13	ProcessingBW.....	38
Appendix C	Procedures to change pulse program values	38

Appendix D	An introduction to Prospa programming	39
D.1	Data types	40
D.2	Variables.....	41
D.3	Control statements	42
D.4	Procedures	42
D.5	Commands	43
D.6	Debugging	43
D.7	Expressions.....	43
D.8	Matrices and vectors	43
D.9	User interface objects.....	44
D.10	Using the Prospa text editor	45
D.11	Additional help.....	45

1 Introduction

This manual will help you to understand and write pulse programs for the SpinsolveExpert application.

The NMR pulse program controls a series of electronic modules in the spectrometer in such a way as to generate the desired NMR signal. At a minimum, this means producing radio frequency (RF) pulses of a specific amplitude, phase, frequency and duration at precise times, and then after a certain delay the NMR signal is captured and returned to the computer for processing and display. In addition there may be control of digital (TTL) levels and magnetic field gradients. If good quality data is to be collected, all of these functions must be applied at very specific times, with high precision and stability.

The Spinsolve spectrometer uses a DSP (digital signal processor) to control the timing and destination of the various signals. A short program runs on the processor to perform this. This program must be first written in a high level language and then translated (automatically) into the required machine code. Because the DSP can only perform a single function at a time it is necessary to write the code in this way:

```
Perform an action
Wait a certain time
Perform next action
Wait a certain time
...
```

Much of the complexity of coding the sequence comes from determining how long to wait between actions.

The code for translating the high level language used for pulse programming is defined in two DLLs (dynamic linked libraries) called UCSPPRun.dll and UCSLockPPRun.dll which are stored in the Prospa DLLs folder. (UCS stands for Ultra Compact Spectrometer – an early name for the Spinsolve). Each DLL includes translation information for some 50 commands used for controlling the spectrometer. However of these only a handful are commonly used. These are:

```

acquire .. collect NMR data
delay .... wait a certain number of microseconds.
wait ..... wait a certain number of microseconds (long version)
pulse .... generate an RF pulse (or pulses) of specific amplitude, phase,
           duration and frequency.
loop ..... repeat a section of code a number of times.
endloop .. define the end of the looped region
shim16 ... set the current in one of the spectrometer shims.
txon ..... switch on the RF
txoff .... switch off the RF

```

A full list may be found in Appendix A. All pulse programs are surrounded by two other commands `initpp` and `endpp`

```

initpp

...

endpp

```

You can see more complete information about each command by typing

```
> help command_name
```

at the common line prompt or by pressing F1 when the insertion point is on the command name (this is valid for all Prospa commands).

When a pulse program is generated on the PC, each of these commands will produce a block of DSP assembly language code which will be stitched together by the `endpp` command and saved to a file for later compilation. The `initpp` command has an argument which specifies the filename for the pulse program output.

The parameters for each command can be of two kinds

- Fixed – which means the command will always use this parameter value e.g. `pulse(1, 16383, 0, 10)`. This will produce a full power pulse on channel 1 with phase 0 for 10 μ s using the currently defined transmitter frequency. In this case you may need to know the conversion factor for the argument value from user, to digital units.
- Variable – in this case the parameters are variable names which will later be given the desired values e.g. `pulse(1, a1, p1, d1)`. The names of these variables are important – the first letter corresponds to the type of the parameter according to this table

```

aXXX ... An RF pulse amplitude in dB.
pNNN ... A phase in degrees/90
dXXX ... A delay in microseconds
fXXX ... A frequency in MHz.
nXXX ... An integer number
tXXX ... A table of numbers

```

Here XXX can be any valid Prospa variable name or an integer and NNN is an integer starting from 1. The phase numbers should be sequential (i.e. p1, p2, p3)

When a pulse program is run, the compiled code is sent to the spectrometer and stored in a specific location (normally p:0x2000, where 0x means a hex number). If a command expects a variable, it will have a reference to another location in memory where this is stored. This is called the pulse program parameter block. This starts at location x:0x0000. Note that the DSP has 3 memories x, y and p. i.e. two data and one program memory. So this syntax means; x memory, address 0. This parameter block updated just before the program starts and may be updated between scans if you need to change pulse parameter values. The order of the parameters in the parameter block will match the order found in the sequence.

So for example, with the following pulse and collect sequence

```
pulse(1, a1, p1, d1, f1)
delay(d2)
acquire("overwrite",n1)
```

the parameters will be stored in this order, from address x:0000

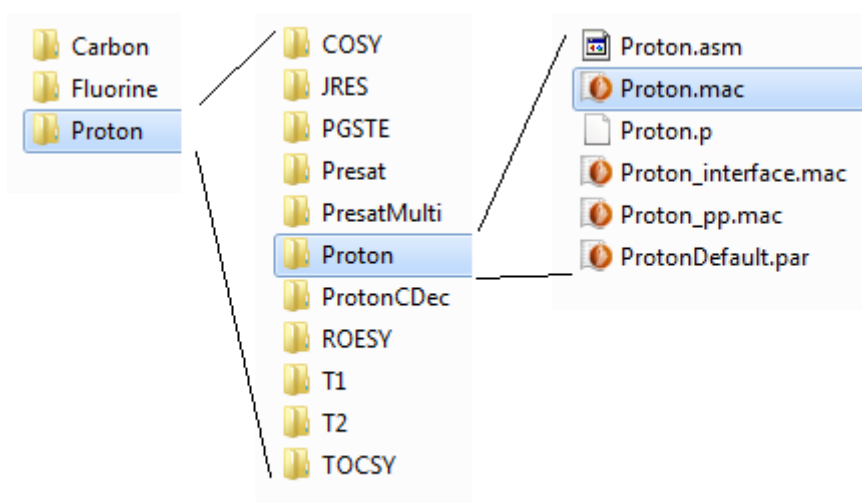
```
x:0000
...
system parameters
...
a1 ... pulse amplitude (a 14 bit number for amplitudes from -85 to 0 dB)
p1 ... pulse phase (an 18 bit number for phases from 0 to 359.999 degrees)
d1 ... pulse duration (a 24 bit number as a multiple of 20 ns)
f1 ... pulse frequency (two 16-bit words)
d2 ... pulse-acquisition delay (a 24 bit number as a multiple of 20 ns)
n1 ... number of points to acquire (a positive 24 bit integer)
```

The system parameters are present for all sequences and includes things like the default transmit and receive frequencies, the receiver amplifier gain and the digital receiver parameters.

The use of special first characters means the system automatically knows how to convert from user units (e.g. dB for RF amplitude) to system units (in this case a 14 bit integer).

2 File organisation

Experiments are typically grouped together, first based on the nucleus and then on the experiment name. Several files are required to run an experiment on the Spectrometer.



All of these files are stored in an experiment folder – ‘Proton’ in this case.

Each of these files has a particular purpose (for other experiments the name ‘Proton’ will be replaced by the experiment name):

Proton.mac The experiment control macro. This contains the code to start the experiment, collect and display the data and also includes loops to accumulate scans or change parameters in a multidimensional experiment.

Proton_interface.mac .. this defines the experiment user interface as it will appear in the SpinsolveExpert application. It defines which parameters are visible, how the plot layout is organised and what post processing options are available.

Proton_pp.mac this defines the pulse program and the visible user parameters. It includes a table which defines the relationship between the user parameters and pulse program parameters. It also defines the phase cycling matrix for the experiment. This macro is only executed during the compilation phase.

Proton.asm this is generated by the pulse program compiler and is an assembly language representation of the pulse program found in Proton_pp.mac. This is not used after the pulse program is compiled and is kept for reference only.

Proton.p this is the machine level file generated from Proton.asm. This file is used to program the DSP and is reused each time the experiment is run.

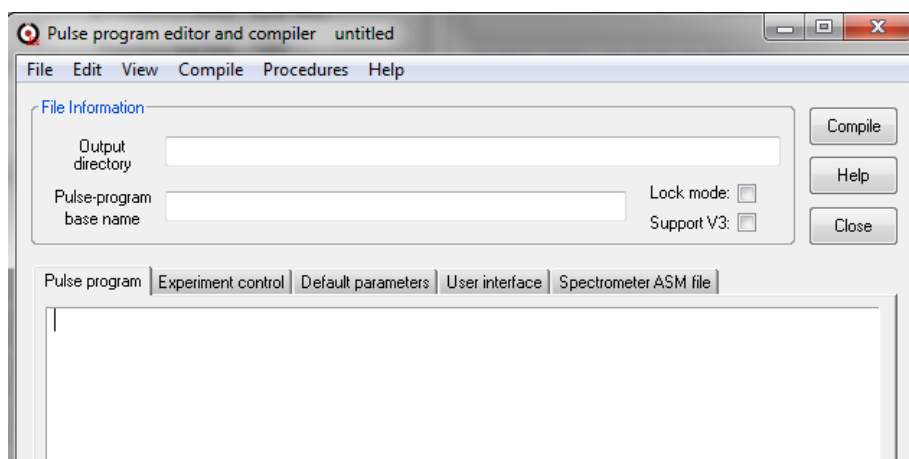
ProtonDefault.par this contains most of the parameters used in the experiment with sensible values which should allow the experiment to run ‘out of the box’. Notice that some the parameters here such as B1 frequency,

pulse lengths and powers (so called common parameters) will be overwritten after the file is loaded into Expert since they depend on the particular Spectrometer you are using. This latter information is read from a FLASH memory block stored on the Spectrometer DSP.

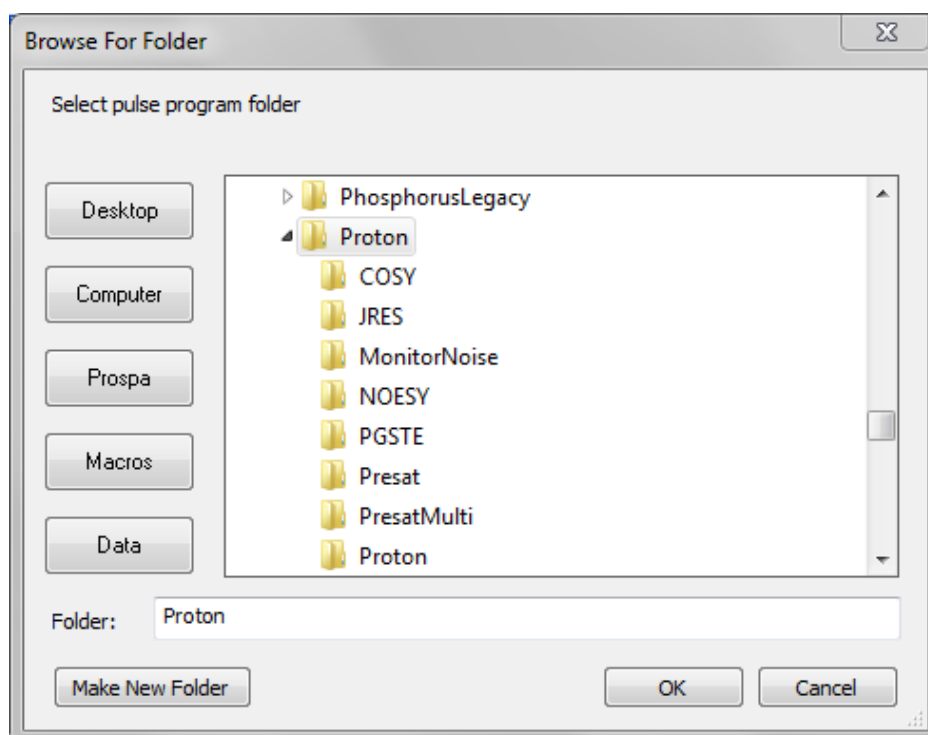
3 The Proton experiment examined

To make this clearer we will now look at the most basic experiment, 'Proton', in more detail.

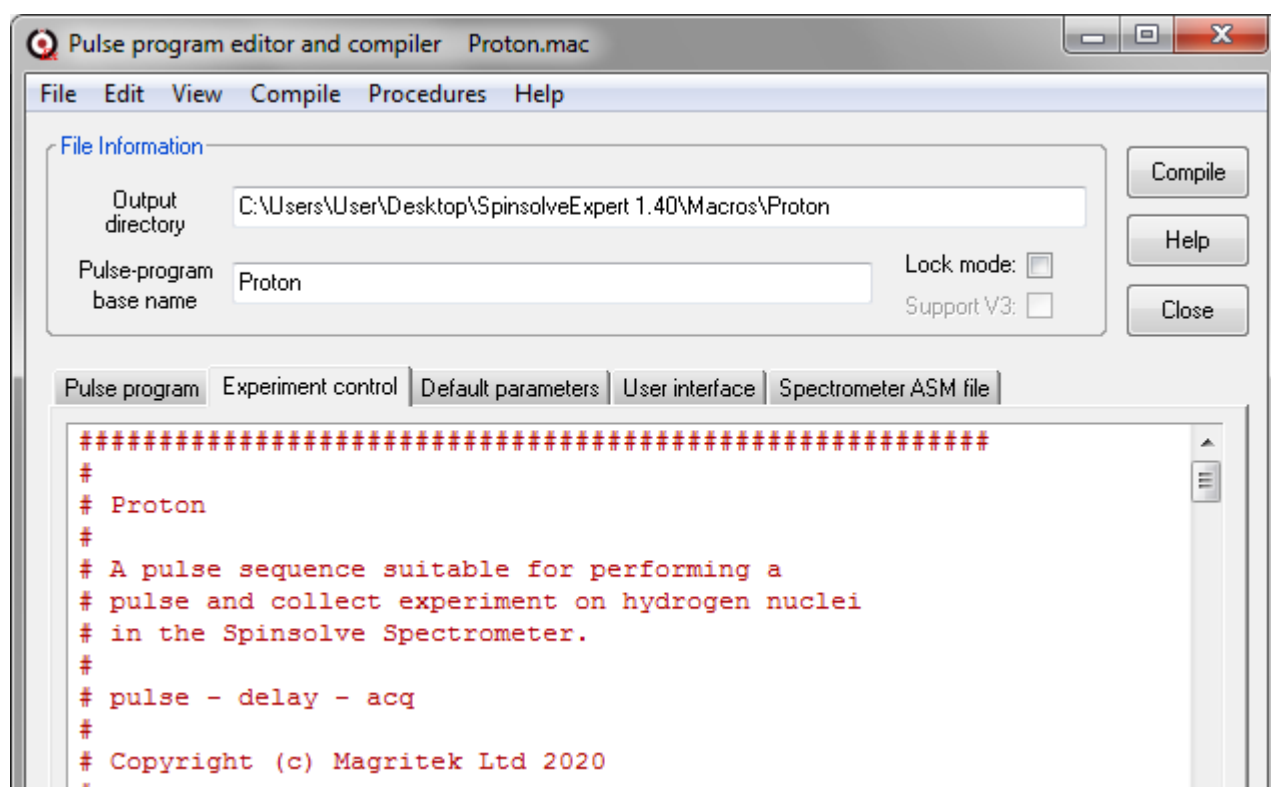
To view the various experiment files, select the experiment from the appropriate menu (e.g. 1H) holding down the shift key at the same time. Alternatively if the experiment is visible in the history or batch lists press the enter key. Finally you can also access the pulse program editor from the options in the Program menu. If you choose the option 'Open pulse program editor' then it will open with a blank interface.



Then you can select the pulse program to load from the File->Open Pulse Program menu option (Ctrl+O).

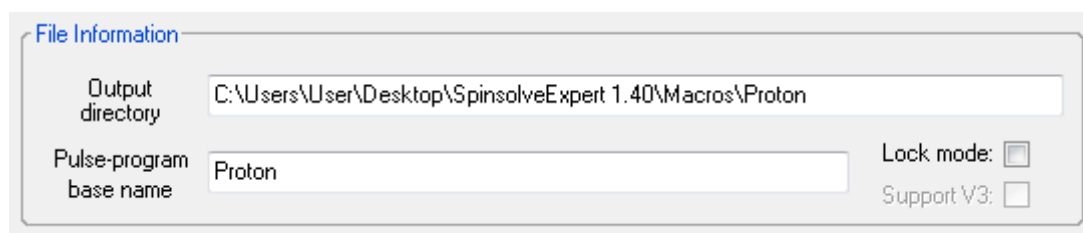


With the Proton folder selected, press the OK button or press Enter. By default this will open the Experiment control macro 'Proton.mac'.



The other important macros associated with the experiment are accessible from the different tabs above the editor. The name of the currently displayed file is visible in the title bar.

Two important file locations are given; the folder the pulse program files are in (Pulse program base name) and the parent directory (Output directory).



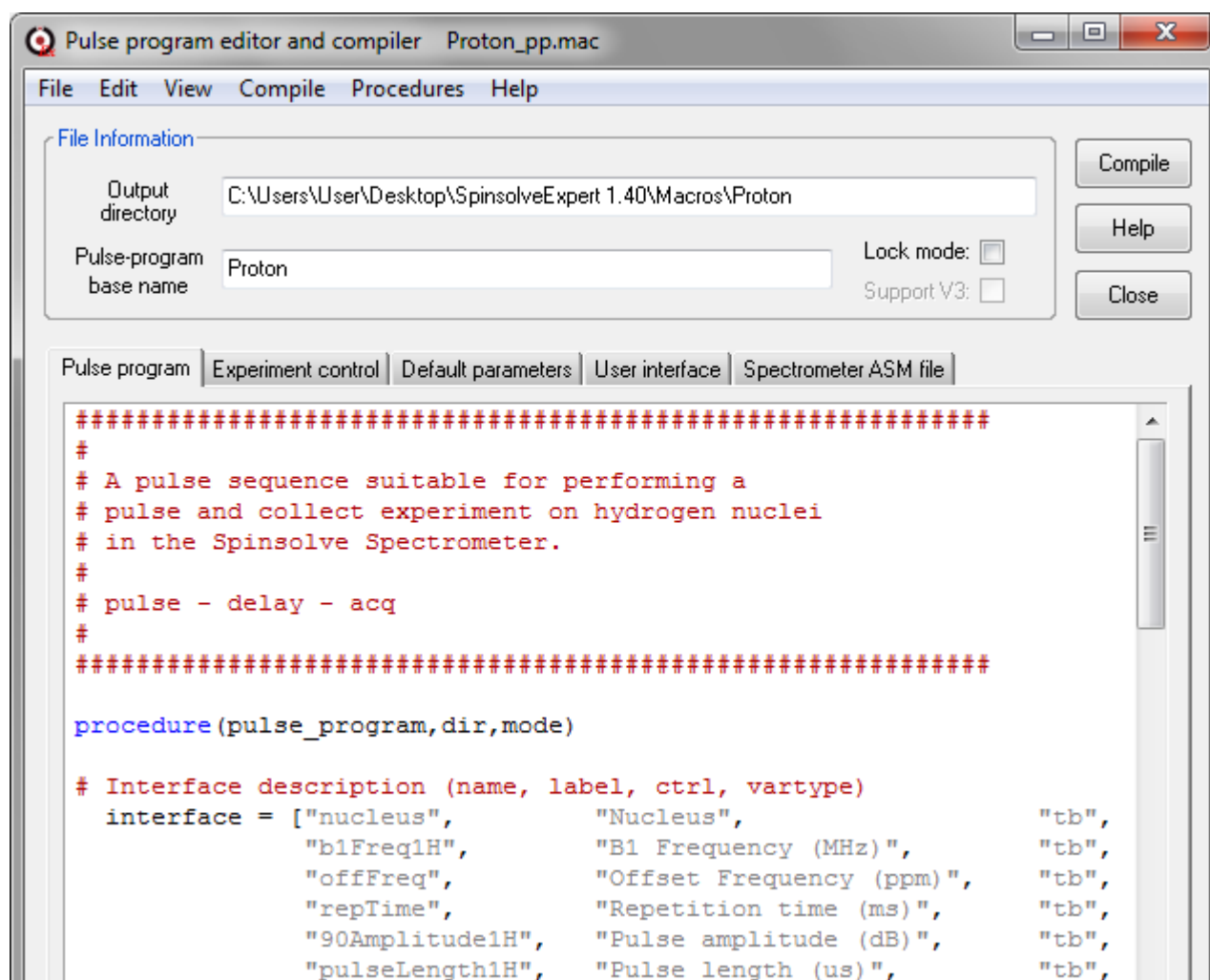
The 'File Information' dialog box contains the following fields and controls:

- Output directory:** A text field containing the path `C:\Users\User\Desktop\SpinsolveExpert 1.40\Macros\Proton`.
- Pulse-program base name:** A text field containing the name `Proton`.
- Lock mode:** A checkbox that is currently unchecked.
- Support V3:** A checkbox that is currently unchecked.

The Lock mode check box is used when compiling lock associated pulse programs (which for most users will be never), and Support V3 is a legacy mode user interface which was similar to that provided for the Kea spectrometer. By default this is disabled.

3.1 The Pulse Program Macro

We will start by looking at the pulse-program macro. Click on the pulse program tab



This macro contains a single procedure call `pulse_program`. It has two argument, 'dir' which is the location where the pulse program will be processed and 'mode' which is the particular

pulsing mode. This will always be '1', standing for channel 1 (the proton channel). 'mode' is a carry-over from the Kea spectrometer which could have internal or external RF amplifiers. For Spinsolve applications you won't need to use 'mode' and should explicitly specify which channel you wish to access.

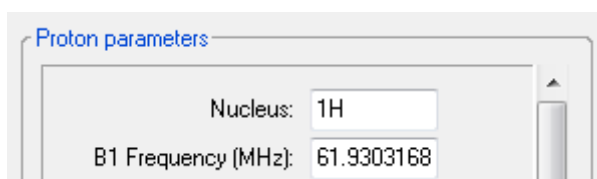
The procedure is broken into several parts; the interface description, the relationship table and optional group interface, the variables list, the pulse program and the phase cycling matrix. We will look at each of these in turn.

3.2 The interface description

The interface is described by a Prospa 2D list¹, each row of which describes one user interface element and label. Note that the last symbol on each row (except in the very last row) must be a semicolon. As an example, the first two rows in Proton are shown below:

```
interface = ["nucleus",      "Nucleus",      "tb",      "readonly_string";  
            "b1Freq1H",    "B1 Frequency (MHz)",  "tb",      "freq";
```

These define the nucleus and the B1 NMR frequency for protons. These generate the first two entries in the Expert Proton parameters user interface.



For each parameter, the interface list entries are; variable name, interface label, type of user interface element (or control) and some special information about the entered data (data type). The abbreviations in column 3 are defined below.

```
tb ..... a text entry field  
tm ..... a text menu (a text entry field with a pull down menu)  
cb ..... a check box  
rb ..... radio buttons  
bt ..... a standard button  
dv ..... a line with a label to divide controls into groups
```

Note that 3 letter symbols are sometimes used, but they are only relevant in V3 interface (e.g. 'w' makes the control wider, however all Expert text entry fields are already wide).

The data type field can take a number of different parameters and formats:

```
double ..... a general double precision floating point number.  
float ..... a general single precision floating point number.  
freq ..... we expect the entry to be a frequency in MHz. This makes  
            it double precision and it must fall within a certain
```

¹ A 1D list can be used but it requires two addition parameters per row which are only used in the V3 legacy interface. Most experiments still use the 1D list.

	range of values as define in the Spinsolve preference dialog.
integer	a general positive integer number [1 ... 1e8].
ldelay	a long delay. For use with the wait command. This can range from 2 μ s to 167 seconds. Units default to microseconds.
ldelayms	a long delay ² . For use with the wait command. This can range from 2 μ s to 167 seconds. Units default to milliseconds.
other	the same as float.
pulseamp	a pulse amplitude in dB. This number can range from -85 (no power) to 0 (maximum power). Note that this is a logarithmic scale with a difference of 6 dB; ideally corresponding to twice the amplitude or quadruple the power.
pulselength	a pulse length in μ s. This can range from 0.5 to 1000 μ s.
readonly_string .	a string which cannot be changed by the end user.
reptime	a repetition time in milliseconds. This cannot be less than 1 ms.
sdelay	a short delay ² - for use with the delay command. This can range from 2 μ s to 327.67 ms. Units default to microseconds.
string	a text string

If used in text-boxes, the generic numeric types `integer`, `float` and `double` can also have an allowable range value as a two element array e.g. `float, [1,100]` which means only numbers between 1 and 100 inclusive are acceptable. Alternatively if the control is a text menu then the array is interpreted as the various menu options e.g. `float, [1,2,3,4,5,6,7,8,9]`

Some control types require additional information in the data type field:

checkboxes	the data type should always be a string and the two options (unchecked and checked) are separated by a comma e.g. "no,yes"
radiobuttons ...	the data type should always be a string and the various options are separated by a comma e.g. "18,opt1,opt2,opt3,Option1, Option2, Option3". This will generate 3 radio buttons with variables names <code>optX</code> and labels <code>OptionX</code> . The first number is the vertical spacing in pixels between the buttons.
textmenus	if the allowable menu options are strings then you should supply a Prospa list e.g. <code>["xy\","yx\","xz\","zx\","yz\","zy\"]</code> . Note that because the data type is a string, the menu sub-strings must have their quotes escaped with a backslash. As mentioned above numerical arrays can be supplied if the options are all numbers.
buttons	the type in this case is a string which contains the callback procedure name for that button. e.g. "zeroShims". The callback procedure must in the Experiment-Control macro.

² See the pulse program commands section.

3.3 The relationships table

As noted previously, the variables used in the pulse program description must conform to strict naming conventions. Also they are typically not the same as the variables we wish to expose to the end user. For this reason we define a relationships table which defines each pulse program parameter in terms of the user interface parameters. In the case of the Proton experiment these relationships are quite simple

```
relationships = ["nDataPnts = nrPnts",  
                "a90Amp    = 90Amplitude1H",3  
                "d90Dur    = pulseLength1H",  
                "dAcqDelay = ucsUtilities:getacqDelay(d90Dur,shiftPoints, dwellTime)",  
                "O1       = offFreq",  
                "totPnts  = nrPnts",  
                "totTime  = acqTime"]
```

Each entry in the table is a string which defines a pulse program variable. Note that it can also define other variables which are used throughout the software. In this case O1 the offset frequency, totPnts; the total number of points collected (which for CPMG experiments may be more than the nrPnts, the number of points collected per echo), and similarly the totTime which might be longer than the acqTime (the case for the CPMG).

This table also shows an example of a function call to update a parameter – in this case the acquisition delay, which depends on several other parameters if we wish to avoid first order phase corrections. Note that we need to include the file name in addition to the procedure name when including procedures in the relationship table. i.e:

```
file_name:procedure_name(arguments)
```

Note that some parameters used here and in other relationship tables are defined elsewhere in the Expert code. These include

acqTime the ideal duration of the data acquisition ($\text{dwellTime} * \text{nrPnts}$) in milliseconds.
bandwidth the bandwidth of the data acquisition (in kHz) Equal to $1000/\text{dwellTime}$.
bandwidth2 .. the bandwidth of the second dimension in a 2D experiment (in kHz).
dwellTime the sampling interval in microseconds.
nrPnts the number of points to collect during the acquire command.
pgo the pulse gate overhead delay. This is defined in the Spinsolve preferences dialog and defaults to 5 μs . It is the time between the start of an RF pulse command and when the pulse actually appears.
rxLat receiver latency. The delay between data appearing at the input and output of the receiver (in microseconds).
x/y/zshim the shim values found for the linear gradients. These are updated after a shim experiment is performed. These are required when crusher gradients are applied.

³ Note that Prospa variables can start with a number!

3.4 The group list

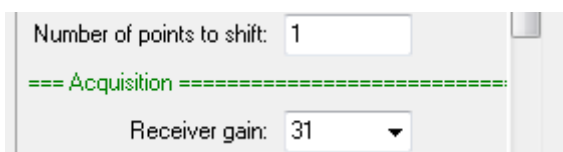
It is possible to define all the graphical user elements in the interface list and that is the case with the Proton sequence (although most are commented out currently). However if you wish to minimise the number of elements in the list you can define a 'groups' list. These are predefined groups of controls which will be included with a single name. e.g. the Proton interface is included with the following list

```
groups = ["Pulse_sequence", "Progress", "Acquisition",  
         "Processing_Std", "Display_Std", "File_Settings"]
```

where the first entry are those items defined in the interface list. The name for this list in older versions of Expert was 'tab' and refers to the legacy interface (V3) which grouped controls into tabs. In the case of the Expert interface, tab groups are separated by the names of the tab. The equivalent control in the user interface list is the divider (dv) e.g

```
"" ,      "Acquisition", "dv", "";
```

Note that the divider control has no associated variable or data type. It produces the following result in the user interface:



Control groups are define in the following folder

\$appdir\$\Macros\UCS-PP\Tabs\AlternateInterface

For example, the Display_Std interface macro looks like this

```
procedure(interface_description)  
  
  interface = [  
    "usePPMScale", "Use ppm scale?", "cb", "string", "no,yes",  
    "dispRangeMinPPM", "Minimum ppm value", "tb", "float", "[-2000,2000]",  
    "dispRangeMaxPPM", "Maximum ppm value", "tb", "float", "[-2000,2000]",  
    "dispRange", "Display range (Hz)", "tb", "float", "[0,2e6]"  
  ]  
  
endproc(interface, "Display")
```

Note that currently, control group interfaces use 1D lists. An option to use 2D list will be included in the future (2D lists allow for better error checking).

For a description of the supplied group lists please see Appendix B. Of course you can also add your own groups using the supplied ones as examples.

If you are not going to use groups then you don't need to include this list.

3.5 The variable list

This is a list of the pulse program variables that you wish to modify during the experiment. For example if you are performing an experiment where the RF pulse amplitude is varied you would include the name of the amplitude variable, e.g. in the Proton experiment this might be `a90Amp`. In this case you would write;

```
variables = ["a90Amp"]
```

If there are no variables to modify then make a single empty element i.e. `[""]`.

3.6 The pulse program

This is the core of this file. These commands describes the pulse program which will be run on the spectrometer. They must be surrounded by the `initpp` and `endpp` commands.

```
# Pulse sequence
initpp(dir)                                # Define compile directory and clear parameter list

delay(5000)                                # Wait 5 ms, allowing time to finish lock scan
pulse(1,a90Amp,p1,d90Dur)                  # RF pulse on channel 1 with phase p1
delay(dAcqDelay)                           # Pulse - acquire delay
acquire("overwrite",nDataPnts)            # Acquire FID

parList = endpp()                          # Combine commands and return parameter list
```

These commands are rather special – unlike other Prosipa commands they don't execute the action immediately, but build up the pulse program in an intermediate assembly language file which will be compiled and executed later on the spectrometer processor. The intermediate language file is stored in the directory 'dir' passed to the `initpp` command. As mentioned previously, either constants or special variables should be used as arguments. The `endpp` completes the file assembly process and returns a list of variable names in the order they appear in the pulse sequence. In this case

```
parList = ["a90Amp", "p1", "d90Dur", "dAcqDelay", "nDataPnts"]
```

Note that inside the pulse program macro, unquoted strings are treated as quoted strings (as they are in the command line interface). This keeps the pulse sequence description cleaner and easier to read. To prevent an external global variable from accidentally replacing an unquoted pulse sequence argument, global and window variables are not accessible from within this macro.

3.7 The phase cycle matrix

Phase cycling is performed by replacing the parameters `p1`, `p2` ... with the phases stored in the `phaseList` 2D matrix before each scan. Each row of the matrix corresponds to a different phase variable while the last row is the acquisition phase. Each column is applied in a different scan. Column 1 for the first scan, column 2 for the second and then wrapping around to the start when we run out of columns. The numbers in the matrix represent multiples of 90 degrees. Fractional numbers can be used for smaller increments. (e.g. 0.5 == 45 degrees)

So in the Proton example,

```
phaseList = [0,1,2,3;    # p1 : Pulse phase  
             0,1,2,3]    # pA : Acquire phase
```

the phase of the RF pulse steps from 0 to 90 to 180 to 270 as the scans are incremented. The acquisition phase is stepped in the same way.

3.8 The returned variables

The various lists generated in the pulse program need to be returned to the compiler. This is done with a parameter list in the `endproc` command.

```
endproc (parList, group, interface, relationships, variables, dim, phaseList)
```

In addition to the lists we have just discussed, there is also a **dim** variable which is used in the legacy interface. This can safely be set to zero if it is not used, however it must still be included for backward compatibility.

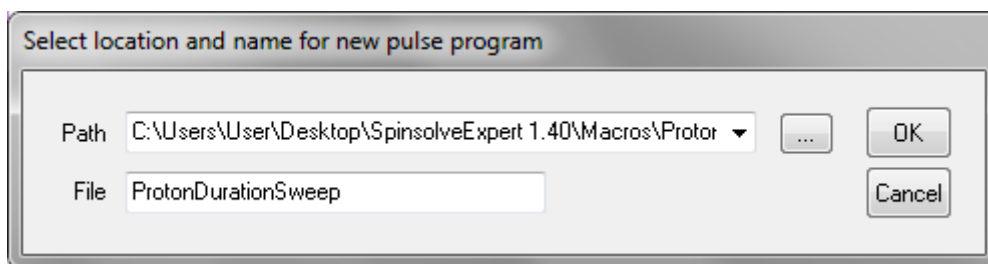
If the group list is not used then you can replace this variable with a null list i.e. `list(0)`. So for example, in the Proton pulse sequence file, if the group list is not used, the return parameters would look like this

```
endproc (parList, list(0), interface, relationships, variables, 0, phaseList)
```

4 Writing a new pulse program

With this brief overview of the Proton sequence we can move on to looking how a new pulse program might be written. Because of the number and complexity of the files involved it doesn't make sense to write an experiment from scratch. Rather you should take an existing experiment and modify it.

Start by opening an experiment which is similar to the one you want to write. For example let's write a pulse duration-sweep experiment to determine the correct 90 degree pulse length. Since the core experiment here is a simple Pulse and Collect, first open Proton in the pulse sequence editor and compiler window. Then in the file menu, select the option "New pulse program from existing file". Select Proton from the dialog window which appears and enter the name for the new experiment – in this case ProtonDurationSweep:



Press OK. This will make a copy of all the Proton files, but with the new name substituted. If you compile this experiment (press the compile button) it will produce identical results to the Proton experiment. The new experiment will now be visible in the Proton (1H) menu. It is recommended that you choose a different path for the new experiment. In this case you will need to add that path to the main menu to access the experiment. This can be done by dragging and dropping the folder corresponding to this path onto the Expert main menu or adding it using the 'Edit pulse program menus' option in the main Edit menu.

Now we can modify the experiment. Since we want to modify the pulse duration, we need to expose the pulse length in the pulse program. To do this, modify the variable list to include the pulse length

```
variables = ["d90Dur"]
```

Save the pulse program (Ctrl+S) and recompile (Press the Compile button or Ctrl+Shift+K)

Before we modify the Experiment Control Macro to sweep the pulse duration let's have a look at the structure of this file.

5 The Experiment Control Macro

This macro contains several procedures:

Main procedure (name will be the same as the macro) ... this provides the option to add the experiment to the user interface or if the shift key is pressed to edit the experiment.

backdoor ... this allows the experiment to be run without using the legacy V3 user interface. Instead a list of parameters is passed. This is used by all Expert experiments.

getseqpar ... get the sequence parameters. This returns the important parameters which were generated when the pulse program was compiled. This includes the relationships table, the variables list, the list of pulse program parameters, the pulse program name and the phase cycling list.

execpp ... executes the pulse program. This contains the commands to send the pulse sequence file (.p) and the list of parameter values to the spectrometer. It runs the sequence and then waits for it to finish. It then reads the collected data, processes it and displays it. It then repeats this process for the desired number of scans, phase cycling the RF pulses and accumulating the collected data as specified in the phase list. When the scans have completed

the data are saved to the folder specified by the Experiment base-path, date and time hierarchy and file comment.

getPlotInfo ... extracts the file names associated with the various plots in the user interface. This is used for saving the data in each plot and also for reloading the last data set when the experiment is reselected.

getFactoryBasedParamters ... some experiments have parameters which are spectrometer specific. This parameters are set in this procedure.

updateProgress ... updates the progress bar during the experiment.

saveProcPar ... saves the processing parameters when the experiment is complete. This is useful if one wants to reprocess the raw FID data in the same way at a later date.

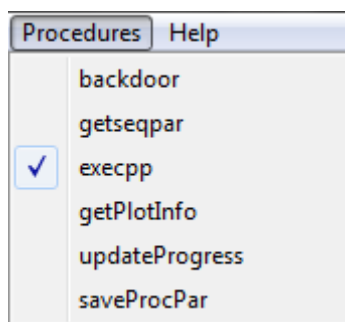
Once an experiment has been selected in the Experiment list and the Run button is pressed the following actions will occur before the execpp procedure is called.

1. A check is made to see if the system is locked and the temperature within acceptable limits. This does not prevent the experiment from running, but updates the Status indicators.
2. The user interface is modified for Run mode. This means the required plots are displayed and various controls are disabled to prevent unwanted user interactions while the experiment is running.
3. The experiment is run in a separate thread to allow the view to be modified and windows resized during the experiment.
4. The experimental parameters are read from the user interface and merged with common and missing default parameters. All parameters are checked for invalid values.
5. The experiment data folder is created and an acquisition parameter file acqu.par is written based on the chosen experimental parameters.
6. The backdoor command in the Experiment Control macro is called.
7. The backdoor macro calls an initialisation procedure (`initAndRunPP`) which combines the user defined experimental and pulse sequence parameters with fixed spectrometer parameters and system preferences to produce the required low level list of pulse sequence parameters which will be passed to the spectrometer processor. For example, this involves converting RF amplitudes from dB to the 14 bit digital value required by the spectrometer's digital synthesiser.
8. The pulse program in binary form is sent to the spectrometer via USB.
9. The execpp procedure in the Experiment Control macro is run executing the pulse program, collecting and processing the data, displaying the data and saving it.
10. A check is again made to see if the system is locked and the temperature within acceptable limits.
11. The interface is enabled and restored to normal operating mode (which allows menu activation and all plot interactions).

5.1 The execpp (execute pulse program) procedure arguments

If the pulse sequence code in the Pulse Sequence macro defines the experiment then the execpp procedure in the Experiment Control macro controls it. Let's look at the Proton experiment macro.

First note that you can jump to different procedures in a macro by using the Procedures menu in the Pulse program editor and compiler window (or any Prospa editor).



The first line of the procedure lists the passed arguments:

```
procedure (execpp, guipar, ppList, pcList, pcIndex, varIndex)
```

execpp is the procedure name, guipar is the list consisting of all the parameters defined in the Proton parameter list plus all the parameters in the relationships list defined in the pulse program macro. Here are some of them:

```
guipar = {  
    90Amplitude1H = 0  
    a90Amp        = 90Amplitude1H  
    accumulate     = "yes"  
    acqTime        = 3276.8  
    b1Freq1H       = 61.9303168d  
    bandwidth      = 5  
    d90Dur         = pulseLength1H  
    ...  
    ycal           = 65536  
    yshim          = -721.995  
    zcal           = 65536  
    zf             = 1  
    zshim          = -1753.47  
}
```

It also includes a number of global variables such as receiver calibrations and gyromagnetic ratios for the various nuclei, as well as other information which may be needed in the macro.

You can list these (or other parameters) yourself by adding the line

```
pr sortlist(guipar)
```

to the execpp code. To see all locally define variables add the line

```
pr local
```

The ppList contains all the generic parameters values required by the spectrometer along with the (highlighted) pulse program parameter values in the same order as they were introduced in the pulse sequence.

0	000F00	18	00000F
1	000500	19	0F4240
2	004E20	20	0000E8
3	000023	21	000001
4	000000	22	000001
5	00002E	23	000006
6	000001	24	0000E8
7	000002	25	0000C8
8	000001	26	000000
9	000A78	27	000000
10	008620	28	003FFF
11	000200	29	000000
12	000004	30	0002CF
13	0068B5	31	0006D8
14	003D40	32	004000
15	000010		
16	00001F		
17	00C4AC		

To print it in this format add the following lines after the start of the execpp procedure

```
for(k = 0 to size(ppList)-1)
  pr "$k$ $hex(ppList [k],24) $\n"
next(k)
```

If you open the Spectrometer ASM file, then at the beginning of the file you will see the corresponding parameter for each of these values. Here we just show the pulse program parameters 28-32

TXA90Amp	ds	1	; 28	- Tx amplitude 90Amp word 0
TXP1	ds	1	; 29	- Tx phase 1
DELAY90Dur	ds	1	; 30	- Delay 90Dur
DELAYAcqDelay	ds	1	; 31	- Delay AcqDelay
NRDataPnts	ds	1	; 32	- Number DataPnts

pcList is the phase cycle list, but now scaled to be in hardware relevant units (90 degrees == 16384)

```
pcList =
  0 16384 32768 49152
  0 16384 32768 49152
```

while pcIndex is their zero based position in the ppList.

```
pcIndex =
```

```
29
```

varIndex is the position of the variables to be modified in ppList. (Not used in Proton)

5.2 The execpp procedure in detail

Now we can break down the various parts of the execpp procedure. The first step is to extract all the guipar list parameters and make them local variables. This is done with the assignlist command

```
# Make all gui parameters available
assignlist(guiPar)
```

Next we allocate space for the collected data. totPnts is the number of complex points in the FID which is also the same as nrPnts in this case

```
# Allocate space for output data
sumData = cmatrix(totPnts)
```

We then determine the axes for the time (FID) and frequency (Spectral) plots

```
# Calculate suitable time and frequency axes
tAxis = ([0:1:totPnts-1]/totPnts)*totTime*1000 # ms
fAxis = [-totPnts*zf/2:totPnts*zf/2-1]/(totTime*zf)*1000 # Hz
```

In addition to totPnts this also uses totTime (== acqTime) and the zero-fill parameter zf. This last parameter was part of the guipar list.

If the user wants to apply a time domain apodization to improve SNR we extract this from the filters macro

```
# Time domain filter
if(filter == "yes")
    flt = filters:get_filter(filterType, "FTFid", totPnts)
else
    flt = matrix(totPnts)+1
endif
```

If not, then the filter is just a constant vector equal to 1.

Next we indicate that we want 2 plot regions, one for the FID and one for the Spectrum. This function returns plot objects we can use for performing the plot.

```
# Get plot regions
(prt, prf) = ucSPlot:getPlotRegions(guiPar, 2)
```

To simplify the generation of axes labels and scales which depend on the ppm/Hz setting and frequency offsets, we call a procedure in the ucSPlot macro to calculate this for us.

```
# Work out frequency axis scale, label and range
(fAxisDisp,fAxisLabel,fRange) = ucsPlot:generate1DFrequencyAxis(prf,
fAxis, blFreq1H, wvPPMOffset1H, offFreq, guipar)
```

The experiment progress bar is then reset and we start accumulating data

```
# Initialise progress bar
:updateProgress(-1,guipar)
```

The core elements of a simple pulse and collect experiment are shown below. First we have the Scan loop:

```
for(scan = 0 to nrScans-1)
```

Next we set the phases based on the phase cycle. Note that this modifies the contents of the ppList variable. It also returns the acquisition phase.

```
# Set phases for this scan
(ppList,pAcq) = ucsRun:setPPPPhase(ppList,scan,pcList,pcIndex)
```

These updated parameters values are then sent to the Spectrometer processor

```
# Send phase parameters to DSP
ucsRun:updatePPPParameters(ppList,guipar,wvPort)
```

The experiment is executed and the collected FID data is returned. Note that the lock is suspended during this process to prevent interference.

```
# Run the pulse program and collect the data
ucsUtilities:suspendLock() # turn lock control loop off
(status,data) = ucsRun:getData(totPnts,guipar)
ucsUtilities:resumeLock() # turn lock control loop on
```

The collected data is accumulated, with the acquisition phase cycle being applied.

```
# Accumulate the data
sumData = ucsRun:accumulate(accumulate,pAcq,sumData,data)
```

The accumulated data is zero filled and then Fourier transformed:

```
# Process data
(phasedTimeData,spectrum,ph0) =
ucsRun:transformData(zeroFill(datacorr.*flt,zf*totPnts,"end"),fAxis,guipar,"fid")
```

And finally plotted

```
# Plot the data
ucsPlot:graphTimeAndFreq(prt,prf,tAxis,datacorr,fAxisDisp,spectrum,scan,guipar, "Time data","Spectral data", "Time (ms)","Amplitude (\G(m)V)", fAxisLabel,"Amplitude")
```

The progress bar is updated

```
# Update progress bar
```

```
:updateProgress(scan,guipar)
```

And then we wait for the end of the repetition time before collecting the next scan

```
# Check timing
check =
ucsRun:checkTimeAndAbort(guipar,t1,scan,pcList,"ignoreLastScan")

if(check == "abort")
    return(0)
elseif(check == "finish")
    scan = scan+1
    exitfor()
endif

next(scan)
```

The Proton experiment also includes some commands to minimise first order phase shifts by setting the acquisition delay carefully and modifying the first few data points to account for the digital filters. For brevity this is not discussed here.

The next step is to save the followed data

```
# Save the data
ucsFiles:savePlot(prt,:getPlotInfo("pt1"),guipar,"noReport")
ucsFiles:savePlot(prf,:getPlotInfo("pt2"),guipar,"simpleReport")
ucsFiles:saveMnovaData(prt,"",guipar,"simpleReport")
```

This saves the FID and spectral plots in a proprietary Prospa format so we can reload the data later, and also in a more basic format 'data.1d' for use with the 3rd party application Mnova. Note the references to the procedure getPlotInfo which returns the filenames for the FID and spectral files.

Finally the processing parameters used are saved for future reference in a proc.par file

```
# Save the processing parameters
:saveProcPar(guipar,ph0,fRange)
```

and then the data is packed up into a structure and returned to the calling program

```
# Pack the data into a structure
result = struct()
result->tx = tAxis
result->ty = sumData/scan
result->fx = fAxisDisp
result->fy = spectrum/scan
result->par = struct(guipar)

# Return result
return(result)
```

5.3 Modifying the execpp procedure to vary the pulse duration

Now we have covered the basic pulse and collect experiment we will consider how to incorporate changing the pulse length parameter as discussed earlier in *Writing a new pulse program*. There we included a non-empty variable parameter list in the pulse sequence file:

```
variables = ["d90Dur"]
```

In the execpp macro this will appear in the varIndex argument as the value 30

```
varIndex = [30]
```

This refers to element 30 in the parameter list ppList:

TXA90Amp	ds	1	;	28	- Tx amplitude 90Amp word 0
TXP1	ds	1	;	29	- Tx phase 1
DELAY90Dur	ds	1	;	30	- Delay 90Dur
DELAYAcqDelay	ds	1	;	31	- Delay AcqDelay
NRDataPnts	ds	1	;	32	- Number DataPnts

To use this parameter we add an extra for-loop outside the scan loop. We also add extra storage for each spectrum recorded:

```
# Calculate the pulse length matrix
minLength = pulseLength1H
maxLength = minLength+nrLengthSteps*stepNr
pulseLengths = linspace(minLength, maxLength, nrLengthSteps)

# Storage for the fids and spectra
fid2d = cmatrix(totPnts, nrLengthSteps)
spectra = cmatrix(totPnts*zf, nrLengthSteps)

# Vary the pulse duration
for(stepNr = 0 to nrLengthSteps-1)

    # Allocate space for output data
    sumData = cmatrix(totPnts)

    # Accumulate scans
    for(scan = 0 to nrScans-1)
```

Note that since we have introduced three new user variables; minLength, lengthStep and nrLengthSteps we need them to the pulse sequence user interface as well:

```
"pulseLength1H", "Min. pulse length (µs)", "tb", "pulselength";
"lengthStep", "Pulse step size (µs)", "tb", "pulselength";
"nrLengthSteps", "Nr. of length steps", "tb", "integer, [1,100]";
```

Also note that we have reused pulseLength1H as the minimum and initial pulse length. Don't forget to recompile after making changes to the *_pp.mac file.

The next step is to save each collected spectrum into the spectra matrix. This is done between the end of the scan loop and the end of the pulse length loop.

```
next(scan)

# Save the FID and Spectrum
```

```

fid2d[~,stepNr] = sumData
spectra[~,stepNr] = spectrum

next(stepNr)

```

The tilde symbol (~) means replace a row. A colon (as in Matlab) can also be used.

We also save the fids to allow export to MNova.

To give some feedback to the user we can plot the accumulated spectra into an additional plot. To do this we need to add the extra plot pt3 in the User Interface macro:

```

procedure(plot_run_layout)

    layout = ["pt1", "pt2"; "pt3"]

endproc(layout)

```

```

procedure(plot_load_layout)

    layout = ["pt3"]

endproc(layout)

```

We also need to add a reference to this third plot in the execpp procedure by changing the number of plots from 2 to 3 and returning another plot name prs:

```

(prt,prf,prs) = ucsPlot:getPlotRegions(guipar,3,wvPP)

```

We can then display the collected data as it is acquired

```

next(scan)

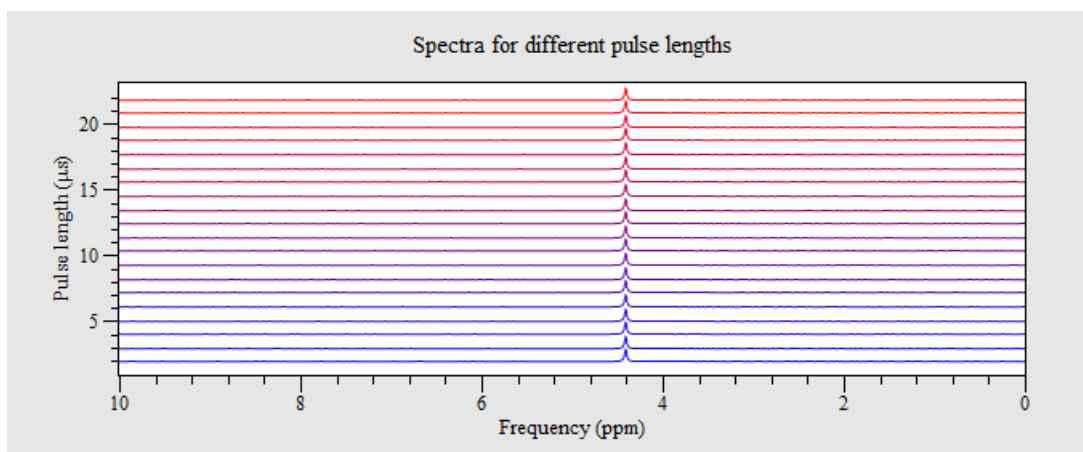
# Save the FID and Spectrum
fid2d[~,stepNr] = sumData
spectra[~,stepNr] = spectrum

StackedPlot(prs, real(spectra), stepNr, fRange, fAxisDisp, "yes",
pulseLengths,"linear", "Frequency (ppm)", "Pulse length
(\G(m)s)","Spectra for different pulse lengths")

next(stepNr)

```

If the pulse sequence is now recompiled and a sensible pulse length range added (2 to 21 μ s in 1 μ s steps) we gain the following result



The first thing we notice is that the amplitude of each spectrum is not changing. That is because we have not changed the pulse length being sent to the Spectrometer. To do this we need to modify the ppList before it is sent to the spectrometer. We do this with the utility procedure `ucsRun:setPPDelay`

```
# Vary the pulse duration scans
for (stepNr = 0 to nrLengthSteps-1)

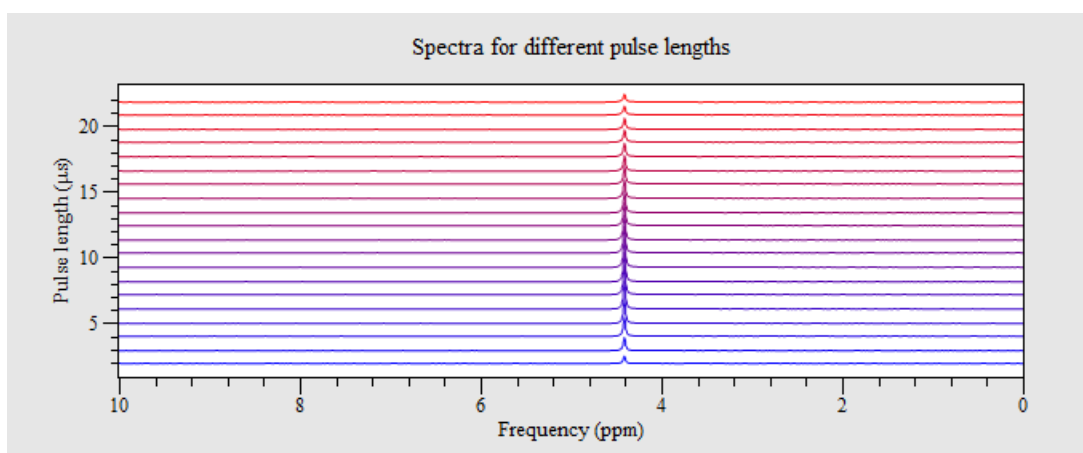
  # Modify ppList with the new pulse length
  ppList = ucsRun:setPPDelay (ppList,varIndex[0],pulseLengths[stepNr])

  # Allocate space for output data
  sumData = cmatrix (totPnts)

  # Accumulate scans
  for (scan = 0 to nrScans-1)
```

If you look in `ucsRun` you will find a number of other conversion procedures which you need for updating different parameters. (See appendix C for a list of these).

After this change we see the amplitude changing nicely



Just one more step is necessary – we need to save this data. Modify the save commands to save the stacked plot and the raw FID data for MNova analysis


```
ucsFiles:savePlot(prs,:getPlotInfo("pt3"),guipar,"noReport")
ucsFiles:saveMnovaData(fid2d,"",guipar,"simpleReport")
```

and in the getPlotInfo procedure make sure we save this data:

```
info = ["pt3","pulseDurationSweep.pt1"]
```

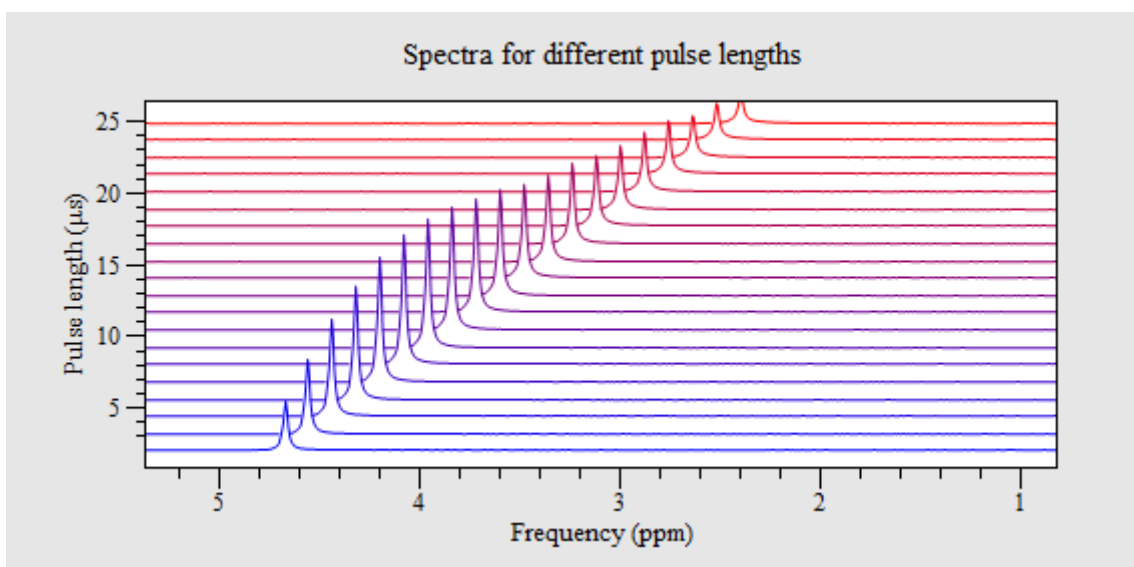
We can also add some post processing tools to the user interface macro to change the stacked plot view , integrate the data and give access to MNova.

```
procedure (processing_controls)

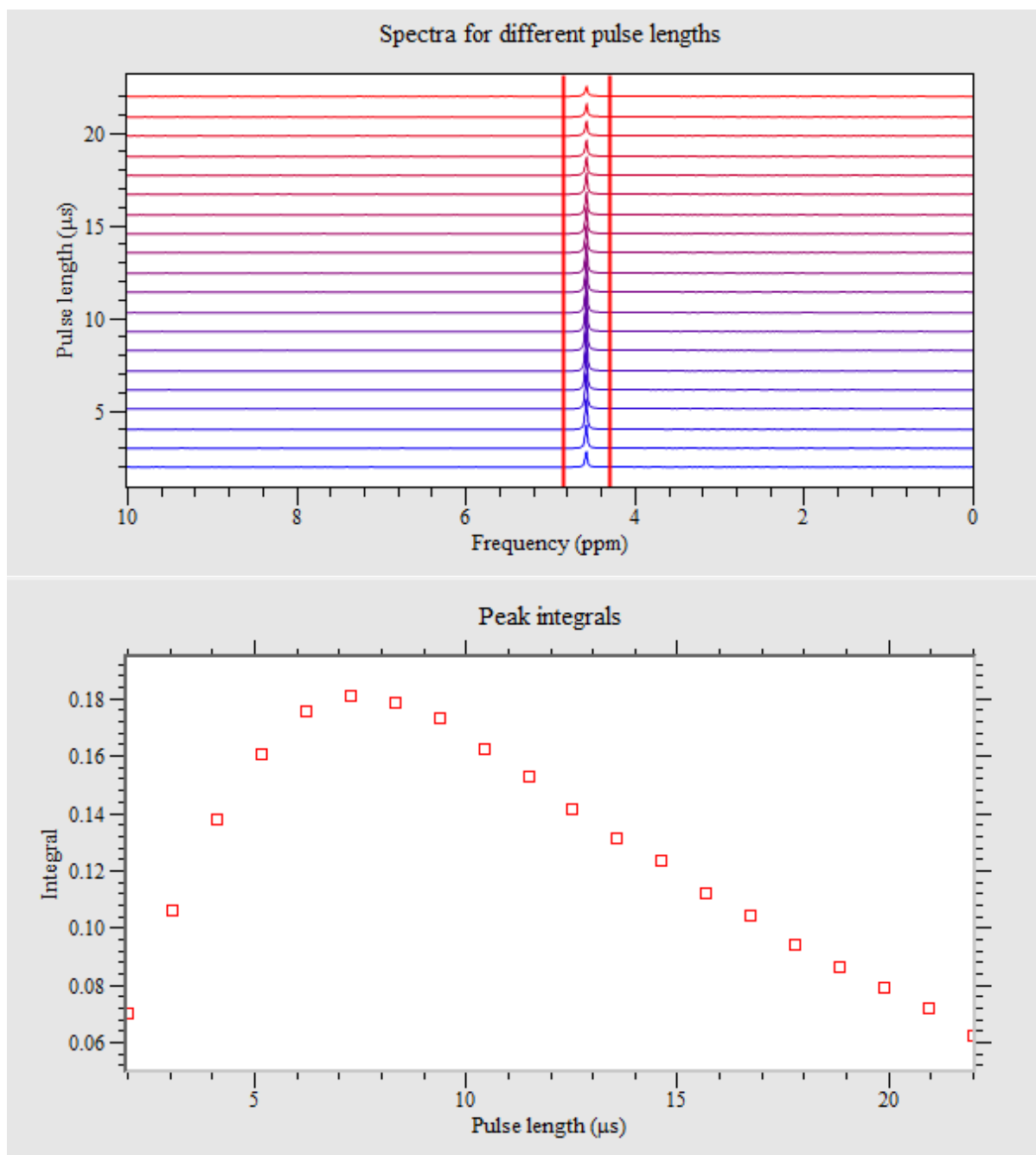
    layout = ["buttonLabel = \"View\\",          "plotName = \"pt3\\",
"macroToRun = \"StackedPlotSetup()\\",
              "buttonLabel = \"Integ.\\",        "plotName = \"pt3\\",
"macroToRun = \"IntegrateRegions()\\",
              "buttonLabel = \"Mnova\\",          "plotName = \"pt3\\",
"macroToRun = \"exportMnova2D(\\\\"pt3\\\\"\\")\\"]

endproc (layout)
```

The View option allows more informative stacked plot displays



While the Integ option allows analysis of the peak integrals:



An important step after writing and testing a new pulse program is to save the last run set of parameters to the default parameter file. This option can be found in the files menu (*Save to default parameter file*). This allows subsequent use of the experiment to have sensible start values. Be careful not to apply this option to the standard experiments!

To give more useful feedback as to the progress of the running experiment the progress bar should be updated correctly. This requires modifications to the `updateProgress` procedure

```
totTime = nrLengthSteps*nrScans*repTime/1000
expTime = (stepNr*nrScans + (scans+1))*repTime/1000
```

Now the progress bar will step correctly.

A completed ProtonDurationSweep experiment is included in the Proton experiment folder for reference.

Appendix A Pulse program commands

Here is a list and a short description of all pulse program commands grouped according to the function type. More details can be found by typing help and then the command name in the command line interface e.g.

```
> help pulse
```

A.1 Acquire NMR data

This command acquires data from the Proton and X-channel receiver and stores it on the DSP memory. No other command can be run during this process.

Blocking acquire

```
acquire(mode, points, [duration])
```

mode ... is one of:

overwrite	data is written to the beginning of memory.
append	data is appended to last data collected.
sum	acquired data is added to previous data.
integrate	acquired data is summed and returned as single point.

points the number of complex points collected. (nx)

duration is a short delay, (dx), (1 μ s to 327 ms), which is applied to the append, sum and add modes. This is the total length of time between the start of data acquisition and the next pulse sequence command.

Non-blocking acquire

This command acquires data from the Proton and X-channel receiver in the background storing it on the transceiver board. It returns immediately so other commands can be run.

```
acquireon(points)
```

points: the number of complex points to collect. (nx)

This command reads in the acquired data obtained with acquireon according to the mode setting copying it to the DSP board.

```
acquireoff(mode, points)
```

mode ... is one of:

`overwrite ...` data is written to the beginning of memory.

`points ...` the number of complex points to collect. (`nx`)

A.2 Clear the DSP data memory

When data is added to memory in the acquire command you should first run this command to ensure it is zeroed.

```
cleardata(number_of_points)
```

`number_of_points ....` number of points in DSP memory to clear (`nx`)

A.3 Delay for a specified time

Short delay

```
delay(duration)
```

`duration .....` is a short delay, (`dx`), (2 μ s to 327 ms)

Long delay

This delay provides a much larger range of values at the expense of more code in the pulse program.

```
wait(duration)
```

`duration .....` is a long delay, (`wx`), (2 μ s to 167 s)

A.4 Frequency modifications

Set the receiver frequency

```
setrxfreq(value)
```

`value ...` frequency to use (`fx`).

Set the transmitter frequency

```
settxfreq(value)
```

value ... frequency to use (f_x).

Or

```
settxfreq(channel, value)
```

channel ... which channel frequency to change ($1/2$).

value frequency to use (f_x).

```
settxfreqs(value1, value2)
```

value1 ... frequency to use for channel 1 (f_x).

value2 ... frequency to use for channel 2 (f_x).

Increment the receiver frequency

```
incrxfreq(increment)
```

increment ... amount frequency should be incremented by (f_x).

Increment the transmitter frequency

```
inctxfreq(increment)
```

increment ... amount frequency should be incremented by (f_x).

A.5 Looping

Start a loop

All commands between loop and matching endloop will be repeated loop_number times.

```
loop(loop_name, loop_number)
```

loop_name unique identifier for the loop (l_x)

loop_number number of times to execute the loop (n_x)

End a loop

```
endloop(loop_name)
```

```
loop_name ..... unique identifier for the loop (lx)
```

A.6 RF pulse production

Several commands can be used to produce an RF output, either as a pulse or a phase, frequency and amplitude modulated waveform.

Simple pulse

Generates a single or dual pulse with specified amplitude, phase and duration

Single channel version

```
pulse(destination, amplitude, phase, duration, [frequency])
```

destination is either 1 (proton channel) or 2 (X channel)

amplitude the pulse amplitude, (ax).

phase pulse phase, (px), – set in the phase cycle array.

duration is a short delay, (dx), (1 μ s to 327 ms).

frequency an optional frequency in MHz (fx).

Dual -channel version

```
pulse(1, amplitude1, phase1, frequency1,  
      2, amplitude2, phase2, frequency2, duration)
```

Parameters as above except destinations are fixed and the duration is common.

Shaped pulse

Generates an RF pulse with modulated amplitude.

```
shapedrf(destination, ampTable,  
          signTable, phase, tableSize, stepTime)
```

destination is either 1 (proton channel) or 2 (X channel)

ampTable an amplitude table (magnitude only), (tx).

signTable a table showing the sign of the pulse amplitude, (tx).

phase pulse phase, (px), – set in the phase cycle array.

tableSize the number of entries in each table, (nx).

stepTime how long each step will last, (dx).

Chirped Pulse

Generates an RF pulse with modulated amplitude and frequency

```
chirprf(destination, ampTable, freqTable, phase, tableSize,  
        stepTime)
```

destination is either 1 (proton channel) or 2 (X channel)
ampTable an amplitude table (magnitude only), (t_x).
freqTable a frequency table (positive only), (f_x).
phase pulse phase, (p_x), – set in the phase cycle array.
tableSize the number of entries in each table, (n_x).
stepTime how long each step will last, (d_x).

Switch on the RF output

Simply switches on the RF output. Use delays to define the duration

```
txon(destination, amplitude, phase)
```

destination is either 1 (proton channel) or 2 (X channel)
amplitude the pulse amplitude, (a_x).
phase pulse phase, (p_x), – set in the phase cycle array.

Switch off the RF output

Switches off the RF output.

```
txoff(destination)
```

destination: is either 1 (proton channel) or
2 (X channel)

A.7 Shim and gradient commands

These commands control the currents in the various shims and where fitted, the gradient.

Shim control

Set a shim value


```
shim16(channel, amplitude)
```

channel the shim channel (1-15) (nx)
amplitude ... the shim value (16 bit signed number) (nx)

Ramp a shim current linearly

```
shimramp16(channel, start, end, steps, duration)
```

channel the shim channel (1-15) (nx)
start the start ramp value (16 bit signed number) (nx)
end the end ramp value (16 bit signed number) (nx)
steps number of steps in the ramp (nx)
duration ... how long each step lasts (dx)

Gradient control

Switch on the gradient (where fitted)

```
gradon(amplitude)
```

amplitude ... the gradient value (a 16 bit signed number) (nx)

Switch off the gradient (where fitted)

```
gradoff()
```

Ramp the gradient current linearly

```
gradramp(start, end, steps, duration)
```

start the start ramp value (16 bit signed number) (nx)
end the end ramp value (16 bit signed number) (nx)
steps number of steps in the ramp (nx)
duration ... how long each step lasts (dx)

A.8 Table commands

These commands control the location of the table index – ie. Which value in a table will be used next.

```
decindex(table)
```

table ... table index to decrement (tx)

```
incindex(table)
```

```
table ... table index to increment (tx)
```

```
setindex(table, index)
```

```
table ... table index to increment (tx)
```

```
index ... value for the index (nx)
```

Appendix B Available control groups

Here is a list of the control (tab) groups available in this version of Expert which are used by some of the experiment and the key feature(s) which differentiate them. These may be found in the folder:

<prospa>\Macros\UCS-PP\Tabs\AlternateInterface

B.1 Acquisition

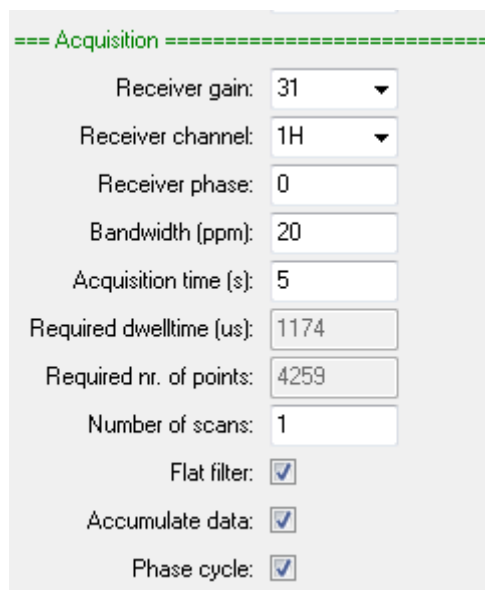
Collect data by specifying a dwelltime (sampling time) and the number of points to collect. This is the natural machine units although not necessarily the most comfortable for the spectroscopist. Note that the bandwidth and acquisition time are automatically updated if the number of points or dwell time is modified.

=== Acquisition =====

Receiver gain:	31	▼
Receiver channel:	1H	▼
Receiver phase:	0	
Number of points:	16384	▼
Dwell time (us):	200	▼
Number of scans:	1	
Flat filter:	☒	
Accumulate data:	☒	
Phase cycle:	☒	
Bandwidth (kHz):	5	
Acquisition time (ms):	3276.8	

B.2 AcquisitionBW

Collect data by specifying a bandwidth in PPM and an acquisition time. Be aware that small bandwidths (or large dwelltimes) may lead to excessive first order phase distortions.



The screenshot shows a configuration window titled "Acquisition" with a green dashed border. It contains several input fields and checkboxes:

- Receiver gain: 31 (dropdown menu)
- Receiver channel: 1H (dropdown menu)
- Receiver phase: 0 (text input)
- Bandwidth (ppm): 20 (text input)
- Acquisition time (s): 5 (text input)
- Required dwelltime (us): 1174 (text input, greyed out)
- Required nr. of points: 4259 (text input, greyed out)
- Number of scans: 1 (text input)
- Flat filter: ☒
- Accumulate data: ☒
- Phase cycle: ☒

Note that the dwelltime and required number of points are automatically updated if the bandwidth or acquisition time is modified.

B.3 AcquisitionBW2D

A 2D version of AcquisitionBW.

B.4 AcquisitionFIR

The same as the standard acquisition list except it provides the option to specify how much FIR decimation should be applied. This is used with sequences which do not apply the FIR filter in the spectrometer hardware but rather delay this until the data is in the PC. This allows linear prediction to be used to minimise baseline and first order phase correction problems by predicting the first few data points in the FID. However it does require that more data is collected. This is determined by the FIR decimation factor.

=== Acquisition ===

Receiver gain:	31
Receiver channel:	1H
Receiver phase:	0
Number of points:	2048
Dwell time (us):	1000
FIR decimation:	4
Number of scans:	1
Flat filter:	☐
Accumulate data:	☒
Phase cycle:	☒
Bandwidth (kHz):	1
Acquisition time (ms):	2048

B.5 Display_Std

Allows the selection of a ppm or Hz scale. The former allows both scale limits to be selected while the latter just allows a width. Zero in the display range (Hz) field displays all the data if 'Use ppm scale' is not checked.

=== Display ===

Use ppm scale?:	☒
Minimum ppm value:	0
Maximum ppm value:	20
Display range (Hz):	500

B.6 Display_2D

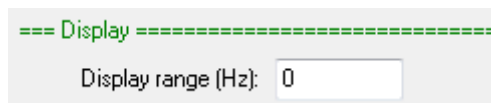
The 2D version of Display_Std which provides control of the 2nd dimension range.

=== Display ===

Use ppm scale?:	☒
Min ppm value f1:	-200
Max ppm value f1:	50
Min ppm value f2:	0
Max ppm value f2:	10
Display range (Hz):	2000

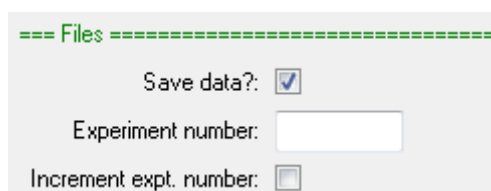
B.7 Display_Hz

Simply allows the display range to be selected in Hz.



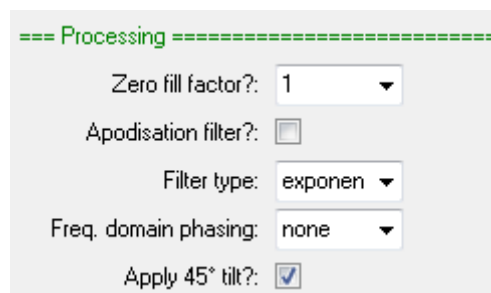
B.8 File_Settings

Here whether data should be saved can be selected, whether experiment numbers should be used and if these number should automatically increment.



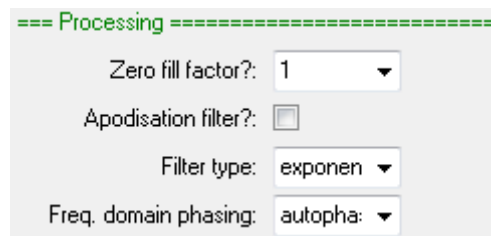
B.9 Jres_processing

The same as Processing_std except there is an option to tilt the 2D data by 45 degrees.



B.10 Processing_std

Provides options to zero fill and apodize the time domain data. Different frequency domain phasing options are also provided.



B.11 Processing_autophase

Just provides an autophase option. Useful with the CPMGInt experiments where time domain filtering is not an option.

B.12 Processing_filter_autophase

The same as Processing_std except it includes an autophase option as well. Useful when you are wanting to phase a time domain dataset e.g. a T1 or a T2 experiment).

B.13 ProcessingBW

Use this option in combination with AcquireBW to work with acquisition time and bandwidth rather than dwellTime and nrPnts. In this case you define the number of spectral points to be equal to or greater than the number of time domain points collected (which won't necessarily be a factor of 2 which is necessary for the Fourier transform used).

Appendix C Procedures to change pulse program values

Many pulse program values are presented in a form which is not suitable for the hardware (dB for RF amplitude, frequencies in MHz, delays in microseconds etc.). This conversion is done automatically for most parameters before reaching the execpp procedure. However when modifying a parameter between scans you will need to explicitly convert the user unit into machine units and update the parameter list. Procedures written in the macro ucsRun can be used for this purpose.

All have the same format:

```
pulse_parameter_list = procedure_name(pulse_parameter_list,
                                     index_of_parameter,
                                     user_unit)
```

In execpp the pulse_parameter_list is the variable ppList. The parameter index is stored in the array varIndex and user_unit you must supply.

Here is the list of helper procedures in `ucsRun`:

`setPPAmplitude` set an RF amplitude (dB) in the pulse sequence parameter array
`setPPDelay` set a short delay (μ s) in the pulse sequence parameter array
`setPPLongDelay` set a long delay (μ s) in the pulse sequence parameter array
`setPPNumber` set a number in pulse sequence parameter array
`setPPPhase` set an RF phase (0-4) in the pulse sequence parameter array
`setPPFrequency` set a frequency (MHz) in the pulse sequence parameter array

An example of using one of these commands is shown below

```
ppList = ucsRun:setPPDelay(ppList, varIndex[0], pulseLengths[stepNr])
```

Note that array index in `varIndex`. This is zero because this is the first (and only in this case) variable in the variable list found in the pulse program

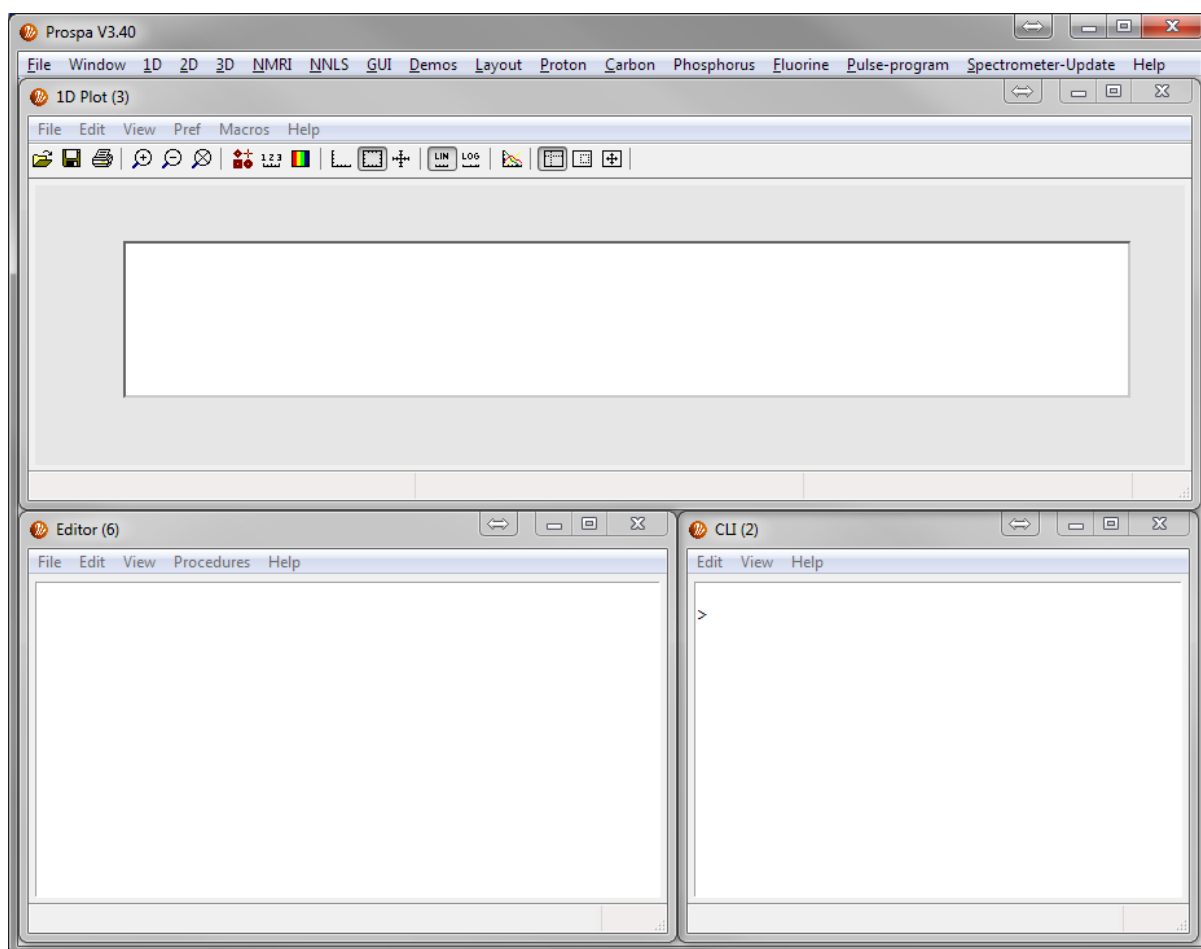
```
variables = ["d90Dur"]
```

If there were two variables and you wanted to modify the second then you would use `varIndex[1]`.

Appendix D An introduction to Prospa programming

The entire `SpinsolveExpert` application is built on top of the Prospa programming environment. This is based around an interpreted language which is reminiscent of Matlab but with some important differences.

When Prospa starts up it runs a script or macro which defines the user interface. If you run Prospa directly from the application folder you will see an interface like this:



This consists of separate windows with simple functionality – a 1D plot, editors and a command line interface. More complex interfaces can be designed (like SpinsolveExpert) which integrate these objects into the one window. (See also the Layout menu for other possibilities).

The scripting language is quite complex consisting of over 400 built in commands and more than 20 data types.

However for the purposes of using the SpinsolveExpert interface only a fraction of these are needed.

D.1 Data types

The most important data types are:

string ... double quoted text based strings which may include escape characters.
Example: "A filename"

list arrays of strings. e.g ["string1","string2","string3"]

list2d ... jagged arrays of lists e.g. ["string11","string12", ... ; "string21","string22", ...]

float ... 4 byte floating point numbers (the default number type) e.g. 3.14159

double ... 8 byte floating point number e.g. 3.1415926535897931d

structure ... a linked list of variables.

vector ... arrays of floats e.g. [1,2,3,4]

matrix ... 2, 3 or 4 dimensional arrays of numbers e.g. [1,2,3,4;5,6,7,8]

objects ... user interface objects such as plots or text boxes. These can be interrogated to extract or set various parameters.

D.2 Variables

Data values are stored in variables. This is done using an assignment statement e.g.

```
myVector = [1,2,3,4]
```

Note that vector name can be any length (although shorter names are interpreted faster), are case sensitive and may include numbers as long as the resultant name is not a valid number. (So 1Pulse is a valid variable name!)

Variables can be collected together in structures. To define a structure use the syntax

```
s = struct()  
s->pi = 3.1415926  
s->vec = [1,2,3,4,5]
```

or more simply

```
s = struct(pi=3.1415926, vec=[1,2,3,4,5])
```

in a procedure, local variables can be converted to a list using the command

```
lst = mkparlist()
```

This takes all local variables (see below) and packs them into a string list. Note that you can't include objects or 3D or 4D matrices in list. Other matrices should be small. The alternative is to use a structure.

To convert a string list into local variables use the reverse command

```
assignlist(lst)
```

Variables defined on the command line interface have global scope (i.e. they are accessible everywhere.) Variables defined inside a procedure (see below) have local scope and are only

accessible from within the procedure. It is also possible to have variables with window scope which are accessible from all children of the parent window.

D.3 Control statements

Prospa supports the following control statements. Syntax is fairly standard.

```
if-elseif-else-endif
```

```
while-exitwhile-endwhile
```

```
for-exitfor-next
```

Please refer to the Prospa documentation for details (type F1 when the cursor is on the command).

D.4 Procedures

Prospa scripts are typically broken in procedures (i.e. functions) which are blocks of code surrounded by the statements

```
procedure(name, arguments)
...
endproc(return_values)
```

Note that multiple values can be returned from a procedure. To exit early from a procedure use the return command.

To call a procedure from within the current file (which should have the extension .mac – for macro or .pex for prospa executable macro), use the following syntax

```
result = :myProc(arg1, arg2)
```

To call a procedure in another file just add the filename

```
result = filename:myProc(arg1,arg2)
```

The extension need not be included as .mac (or .pex) is assumed.

To return multiple values place them in parenthesis

```
(result1,result2) = :procName(arg1,arg2,arg3)
```

All variables defined in a procedure have local scope i.e. they are not visible outside the procedure.

Please refer to the Prospa manual for more details about using procedures.

D.5 Commands

Some 500 predefined commands are included in the Prospa package. A list can be obtained using the `listcom` command. Check the help viewer accessible from the Help menu to see a list of these.

The syntax is exactly as it is for procedures

```
result = command(arg1, arg2, ...)
```

or

```
(result1,result2 ...) = command(arg1, arg2, ...)
```

These commands include functions to manipulate data types, access the file system and build user interfaces.

D.6 Debugging

You can test the various commands and syntax by typing them into the command line interface. Use the print (`pr`) command to print results. e.g.

```
> a = 2^4
> pr a

a = 16
```

D.7 Expressions

Commands, variables and procedures may be combined in expressions where the syntax is valid. e.g.

```
magnitude = sqrt(x^2+y^2)
```

Standard mathematical precedence is followed. Note that if a command with multiple return values appears in the expression only the first return value is used. If you want to extract the *n*th returned value use the *retvar* command (1 based).

D.8 Matrices and vectors

Both real and complex vectors and matrices can be defined. To define a blank matrix of dimensions 1,2,3,4 use the following syntax

```
mat = matrix(dim1, dim2, dim3, dim4)
```

Where *dimx* is the size of the *x*th dimension. Note that for 2 and 3D matrices the dimension order is *x, y, z* **not** *row, col, depth* as it is in Matlab. This syntax is also used when accessing elements

```
result = mat[x,y,z].
```

To define a linear 1D vector use the following syntax

```
result = [start:step:end] (step is optional)
```

or

```
result = linspace(start,end,number)
```

To access the first element use the index **0** while to access last element use the index **-1** (the second to last -2 etc). e.g

```
lastValue= vec[-1]
```

To get the dimensions of a matrix use the size command

```
(width, height) = size(mat)
```

D.9 User interface objects

User interface objects such as buttons, text boxes, plots etc. can be assigned to object variables. They can then be used to access functions and variables embedded in the objects.

Access is done via the arrow operator e.g. for the SpinsolveExpert interface

```
plt1 = pltCtrl1->subplot(1,1)
```

```
plt1->plot(x,y)
```

```
(x,y) = plt1->getdata()
```

The first command plots the x-y pair in the first region plt1 of plot control pltCtrl1 (== pt1). The second command extracts the x-y data from the plot.

To see the commands and variables available for an object just type the object name into the command line interface. Help is also available in the classes topic in the help viewer.

```
> pr plt1
```

```
#### 1D plot-region object ####
```

PARAMETER	VALUE
antialiasing1d.....	"true"
autorange	"true"
axes	object
bordercolor	[230,230,230]
bkgcolor	[255,255,255]
clear	cmd : clear all trace data
dim	"1d"

```

draw ..... "true"
filename ..... "image_section.pt1"
filepath ..... "C:\ ... \Example Data\Plots\"
fileversion ..... 1.4
getdata ..... cmd : extract (x,y) data
grid ..... object
hold ..... "false"
load ..... cmd : load a pt1 file into plot region
parent ..... control: 1
plot ..... cmd : display (x,y) data
position ..... (1,1)
border ..... "true"
save ..... cmd : save plot region contents to a pt1 file
trace ..... object
tracelist ..... cmd : return array of trace id numbers
tracepref ..... cmd : set or get trace drawing preferences
title ..... object
xlabel ..... object
ylabel ..... object
zoom ..... cmd : set viewing limits

```

D.10 Using the Prospa text editor

If you develop Prospa scripts inside the Prospa editor then you will get some additional features compared with writing it in a simple editor such as Notepad.

1. Syntax coloring. Different types of commands are colored differently.
2. Syntax hints. As you type a command the valid arguments are listed in the editor status box.
3. Help. Pressing F1 when the cursor is on a command will display detailed help and often an example.
4. Procedure searching. Using the editor menus or keyboard shortcuts you can jump to and from Procedure code.
5. Indenting and commenting. The editor menu has commands to rapidly comment and indent block of selected code.

D.11 Additional help

Additional Prospa and pulse programming help can be found in the documents:

Prospa programming manual.pdf
 SpinsolveExpert pulse programming manual.pdf
 SpinsolveExpert Pulse Program Commands.pdf

These files can be found in the Prospa install directory in the folder "PDF Documentation".

Help can be found in the Viewer found in the main Help menu.